ПРАКТИЧЕСКАЯ РАБОТА №4
Операции, выражения и операторы.

Цель работы: освоить арифметические операции языка Си, понять разницу между целочисленным и вещественным делением. Изучить операции инкремента и декремента, приоритеты операций и механизм явного приведения типов. Освоить базовую управляющую структуру цикла `while`, передачу аргументов в пользовательские функции и научиться защищать программы от бесконечных циклов при некорректном вводе.

Общее требование: во всех заданиях подключайте `<stdio.h>`, используйте `int main(void)` и завершайте программу оператором `return 0;`. Если в задании указаны `<stdint.h>` или `<limits.h>`, подключите соответствующие заголовочные файлы.

Система вариантов: номер варианта для каждого задания определяется индивидуально по номеру зачетной книжки студента ($N_{зкс}$). Для каждого задания применяется свой коэффициент ($K = 0$) по формулам ниже:

4.1. Вариант = ($N_{зкс}$ % 30) + 1

4.2. Вариант = (($N_{зкс}$ + (K + 7))% 30) + 1

4.3. Вариант = (($N_{зкс}$ + (K + 13))% 30) + 1

4.4. Вариант = (($N_{зкс}$ + (K + 19))% 30) + 1

4.5. Вариант = (($N_{зкс}$ + (K + 23))% 30) + 1

o 4.1. Конвертер единиц измерения

Напишите программу, которая считывает величину в базовых единицах и конвертирует ее в составные единицы, используя целочисленное деление и операцию остатка от деления. Программа должна выводить результат в формате, указанном в таблице для вашего варианта. Для коэффициента конвертации создайте символическую константу посредством `#define` или `const`. Организуйте цикл `while` для многократного ввода значений. Цикл должен завершаться при вводе не положительного числа или сигнала `EOF`. Для вещественных коэффициентов используйте тип `double` и вещественное деление, а для целочисленных – оператор `%`. В комментарии к коду объясните, почему при целочисленном делении дробная часть отбрасывается.

Таблица 4.1.1 – Варианты заданий

№	Величина	Базовая	Старшая	Младшая	Коэффициент
1	Данные	КБ	МБ	КБ	1024
2	Время	мин	час	мин	60
3	Энергия	кДж	МДж	кДж	1000
4	Давление	кПа	МПа	кПа	1000

Таблица 4.1.1 – Варианты заданий

№	Величина	Базовая	Старшая	Младшая	Коэффициент
5	Длина	см	м	см	100
6	Объем	ст.л.	чашки	ст.л.	16
7	Частота	кГц	МГц	кГц	1000
8	Масса	г	кг	г	1000
9	Объем	л	м³	л	1000
10	Мощность	кВт	МВт	кВт	1000
11	Углы	мин	град	мин	60
12	Время	сек	мин	сек	60
13	Длина	мм	см	мм	10
14	Масса	мг	г	мг	1000
15	Объем	мл	л	мл	1000
16	Данные	бит	байт	бит	8
17	Данные	байт	КБ	байт	1024
18	Углы	сек	мин	сек	60
19	Давление	Па	кПа	Па	1000
20	Энергия	Дж	кДж	Дж	1000
21	Частота	Гц	кГц	Гц	1000
22	Скор.	м/с	км/ч	м/с	3.6
23	Мощность	Вт	кВт	Вт	1000
24	Напряжение	мВ	В	мВ	1000
25	Время	час	дни	час	24
26	Время	дни	нед	дни	7
27	Длина	м	км	м	1000
28	Длина	км	мили	км	1.609
29	Масса	кг	т	кг	1000
30	Объем	ч.л.	ст.л.	ч.л.	3

o 4.2. Трансформация данных с приведением типов

Напишите программу, которая считывает два или три значения заданных типов, вычисляет результат по формуле из вашего варианта и выводит его. В формуле необходимо использовать явное приведение типов, чтобы избежать целочисленного деления или потери точности. Также программа должна вывести размеры всех использованных типов с помощью операции `sizeof()`, приведя результат к `unsigned long long` и используя спецификатор `%llu`. Объясните, почему без явного приведения типов результат вычисления был бы неверным.

Таблица 4.2.1 – Варианты заданий

№	Типы входных данных	Формула вычисления
1	`uint8_t a, b`	`(float)(a + b) / 2`
2	`int16_t x, y`	`(float)x / y`
3	`uint16_t w, h`	`(float)(w * h) / 10000`
4	`uint8_t r, g, b`	`0.299*r + 0.587*g + 0.114*b`
5	`int16_t t1, t2`	`(float)(t1 - t2) / t1 * 100`
6	`uint16_t d, t`	`(float)d / t`
7	`uint32_t s, t`	`(float)s / t`
8	`int16_t v1, v2, t`	`(float)(v2 - v1) / t`
9	`uint16_t m, v`	`(float)m * v * v / 2`
10	`uint16_t m, h`	`(float)m * 9.81 * h`
11	`uint16_t u, i`	`(float)u / i`
12	`uint16_t u, r`	`(float)u / r`
13	`uint16_t i, r`	`i * r`
14	`uint16_t q, t`	`(float)q / t`
15	`uint32_t p, t`	`p * t`
16	`uint16_t m, v`	`m * v`
17	`uint16_t f, d`	`f * d`
18	`uint16_t n1, n2`	`(float)n1 / (n1 + n2)`
19	`uint8_t a, b, c`	`(float)(a + b + c) / 3`
20	`int16_t x1, x2, y1, y2`	`(x2-x1)*(x2-x1) + (y2-y1)*(y2-y1)` (приведение к `int32_t`)
21	`uint8_t r, g, b`	`(r + g + b) / 3` (с приведением к `float`)
22	`uint16_t w, h`	`2 * (w + h)`
23	`uint16_t r`	`2 * 3.14159 * r`
24	`uint16_t r`	`3.14159 * r * r`
25	`uint16_t a, b`	`(float)a * b / 2`
26	`uint16_t a, b, h`	`(float)(a + b) * h / 2`
27	`uint16_t d1, d2`	`(float)d1 * d2 / 2`
28	`uint16_t a, b, c`	`(float)(a + b + c) / 2`
29	`uint32_t n`	`(float)n * (n + 1) / 2`
30	`uint16_t n`	`(float)n * (n - 1) / 2`

4.3. Генератор последовательностей

Напишите программу, которая генерирует и выводит последовательность чисел, используя операции инкремента и декремента. Программа должна считать число `N` и вывести первые `N` элементов последовательности, указанной в таблице для вашего варианта. Используйте цикл `while` и переменную-счетчик. Для последовательностей, требующих двух предыдущих значений, объявите две переменные и обновляйте их. Выведите элементы через пробел. В комментарии к коду объясните разницу между постфиксным и префиксным инкрементом и почему в данном задании важен порядок их применения.

Таблица 4.3.1 – Варианты заданий

1	Арифметическая	`a[i] = a[i-1] + 2`	1
2	Треугольные числа	`a[i] = i*(i+1)/2`	i=1
3	Фибоначчи	`a[i] = a[i-1] + a[i-2]`	0, 1
4	Падована	`a[i] = a[i-2] + a[i-3]`	1, 1, 1
5	Геометрическая	`a[i] = a[i-1] * 2`	1
6	Степени пятерки	`a[i] = 5^i`	i=0
7	Сумма натуральных	`a[i] = a[i-1] + i`	0
8	Числа Мерсенна	`a[i] = 2^i - 1`	i=1
9	Арифметическая	`a[i] = a[i-1] - 1`	10
10	Объем чашек	`a[i] = a[i-1] * 16`	1
11	Частота кГц→МГц	`a[i] = a[i-1] * 1000`	1
12	Факториал	`a[i] = a[i-1] * i`	1
13	Трибоначчи	`a[i] = a[i-1] + a[i-2] + a[i-3]`	0, 0, 1
14	Кубы чисел	`a[i] = i * i * i`	i=1
15	Данные КБ→МБ	`a[i] = a[i-1] * 1024`	1
16	Время дни→нед	`a[i] = a[i-1] + 7`	1
17	Мощность кВт→МВт	`a[i] = a[i-1] * 1000`	1
18	Сумма натуральных	`a[i] = a[i-1] + i`	1
19	Давление кПа→МПа	`a[i] = a[i-1] * 1000`	1
20	Квадраты чисел	`a[i] = i * i`	i=1
21	Объем ст.л.→чашки	`a[i] = a[i-1] * 16`	1
22	Частота Гц→кГц	`a[i] = a[i-1] * 1000`	1
23	Масса мг→г	`a[i] = a[i-1] * 1000`	1
24	Углы мин→град	`a[i] = a[i-1] + 60`	0
25	Арифметическая	`a[i] = a[i-1] + 3`	0

1	Арифметическая	`a[i] = a[i-1] + 2`	1
26	Объем л→м³	`a[i] = a[i-1] * 1000`	1

Таблица 4.3.1 – Варианты заданий

№	Тип последовательности	Формула / Логика	Начальное значение
27	Мощность Вт→кВт	`a[i] = a[i-1] * 1000`	1
28	Данные байт→КБ	`a[i] = a[i-1] * 1024`	1
29	Энергия кДж→МДж	`a[i] = a[i-1] * 1000`	1
30	Напряжение мВ→В	`a[i] = a[i-1] * 1000`	1

o 4.4. Накопительный калькулятор

Напишите программу, которая вычисляет математическую функцию с использованием цикла `while` и переменной-аккумулятора. Программа должна считывать параметр `N` (или набор чисел, в зависимости от варианта) и вычислять результат по заданной формуле. Для вариантов 18-22 сначала считайте `N`, затем в цикле считайте `N` чисел. Для вариантов 23-25 используйте цикл для разложения числа на цифры. Выведите результат с точностью до 6 знаков после запятой для вещественных чисел. В комментарии к коду объясните, почему для накопления суммы используется оператор `+=`, а для произведения `*=`.

Таблица 4.4.1 – Варианты заданий

№	Функция	Формула / Логика
1	Сумма первых N	`S = 1 + 2 + ... + N`
2	Максимум	`считывание N чисел`
3	Знакочередующаяся	`S = 1 - 2 + 3 - 4 + ... ± N`
4	Сумма степеней	`S = 1^1 + 2^2 + ... + N^N`
5	Геометрическая сумма	`S = 1 + 2 + 4 + ... + 2^(N-1)`
6	Количество нечетных	`считывание N чисел`
7	Сумма обратных	`S = 1/1 + 1/2 + ... + 1/N`
8	Число Армстронга	`проверка суммы кубов цифр`
9	Арифметическая прогрессия	`S = N*(a1+aN)/2`
10	Факториал	`N! = 1 * 2 * ... * N`
11	НОД двух чисел	`алгоритм Евклида`
12	Сумма кубов	`S = 1³ + 2³ + ... + N³`
13	Сумма цифр числа	`разложение числа`
14	Произведение	`P = 1 * 2 * ... * N`
15	Сумма положительных	`считывание N чисел`

№	Функция	Формула / Логика
16	Сумма квадратов	S = 1² + 2² + ... + N²

Таблица 4.4.1 – Варианты заданий

№	Функция	Формула / Логика
17	Простое число	проверка делителей
18	Сумма факториалов	S = 1! + 2! + ... + N!
19	Совершенное число	сумма делителей
20	Квадраты чисел	a[i] = i * i
21	Сумма нечетных	S = 1 + 3 + 5 + ... + (2N-1)
22	Число сочетаний	C(N,2) = N*(N-1)/2
23	Масса мг→г	a[i] = a[i-1] * 1000
24	Количество четных	считывание N чисел
25	Арифметическая	a[i] = a[i-1] + 3
26	НОК двух чисел	через НОД
27	Обратное число	разложение числа
28	Сумма четных	S = 2 + 4 + 6 + ... + 2N
29	Энергия кДж→МДж	a[i] = a[i-1] * 1000
30	Число размещений	A(N,2) = N*(N-1)

o 4.5. Модуль конвертации с функциями

Напишите программу с функцией, которая выполняет конвертацию по заданной формуле. Функция должна принимать один аргумент типа `double` и возвращать `double`. Используйте `const` для коэффициентов конвертации внутри функции. Организуйте цикл `while` в `main()` для многократного ввода. Используйте проверку `scanf("%lf", &value) == 1` для защиты от некорректного ввода и `EOF`. Выведите результат с точностью до 2 знаков после запятой. Поясните, почему функция принимает `double`, а не `float`, и как возвращаемое значение `scanf` помогает избежать бесконечного цикла.

Таблица 4.5.1 – Варианты заданий

№	Название функции	Формула конвертации
1	c_to_f	F = C * 9/5 + 32
2	min_to_hr	hr = min / 60
3	km_to_mi	mi = km / 1.609
4	w_to_kw	kw = w / 1000
5	m_to_ft	ft = m * 3.281
6	v_to_mv	mv = v * 1000

№	Название функции	Формула конвертации
7	l_to_gal	gal = l / 3.785
8	hr_to_day	day = hr / 24

Таблица 4.5.1 – Варианты заданий

№	Название функции	Формула конвертации
9	c_to_k	K = C + 273.15
10	in_to_cm	cm = in * 2.54
11	k_to_c	C = K - 273.15
12	kg_to_lb	lb = kg * 2.205
13	bit_to_byte	byte = bit / 8
14	cm_to_in	in = cm / 2.54
15	hr_to_min	min = hr * 60
16	f_to_c	C = (F - 32) * 5/9
17	kw_to_w	w = kw * 1000
18	mi_to_km	km = mi * 1.609
19	mv_to_v	v = mv / 1000
20	ft_to_m	m = ft / 3.281
21	day_to_hr	hr = day * 24
22	gal_to_l	l = gal * 3.785
23	lb_to_kg	kg = lb / 2.205
24	day_to_yr	yr = day / 365
25	byte_to_kb	kb = byte / 1024
26	a_to_ma	ma = a * 1000
27	yr_to_day	day = yr * 365
28	kb_to_mb	mb = kb / 1024
29	ma_to_a	a = ma / 1000
30	mb_to_gb	gb = mb / 1024